

CARKIT 2C0 GMSL User Guide

1	Content of release package.....	3
2	Features	4
2.1	Control and configuration interface.....	4
2.2	Raw data streaming	4
2.2.1	Using v4l2-ctl application	4
2.2.2	Memory-mapped stream via a custom application	5
2.3	Control from Matlab host PC	6
3	Software and Hardware Pre-requisites	7
4	Hardware build-up	8
5	Software environment	9
5.1	Custom video driver and device tree	9
5.2	Serializer/De-serializer configuration	11
5.3	iRFE-Jetson.....	11
5.4	Strata	12
6	Start-up	13

The scope of this User Guide is to describe the usage of CARKIT 2C0 together with the Nvidia Jetson AGX Orin and Max9295D-Max9296A serializer-deserializer.

Please see the chapter software and hardware pre-requisites for further information about target system and used software versions.

This user guide describes the current stable release. The system still has limitations, both in hardware and software, and is not intended for productive use and should only be used for evaluation purposes.

Infineon will provide continuous software updates.

Limitations of this release

Please read and acknowledge these limitations carefully. Limitations are planned to be removed in upcoming releases.

- Only one CARKIT 2C0 supported over GMSL2 link
- Radar application cycle is controlled from user space
 - Timing of application cycle by use of user space "sleep" API
 - Results in reduced accuracy
 - The timing will be improved in future versions
- First frame after transition from Low Power to Operation state is not correctly transmitted when using non-continuous clock mode
 - Default examples all use continuous clock mode
- First frame after CTRX reset is not correctly transmitted.
 - Default examples do not read the first frames.

Table of contents

- [Content of release package \(see page 3\)](#)
- [Features \(see page 4\)](#)
 - [Control and configuration interface \(see page 4\)](#)
 - [Raw data streaming \(see page 4\)](#)
 - [Using v4l2-ctl application \(see page 4\)](#)
 - [Memory-mapped stream via a custom application \(see page 5\)](#)
 - [Control from Matlab host PC \(see page 6\)](#)
- [Software and Hardware Pre-requisites \(see page 7\)](#)
- [Hardware build-up \(see page 8\)](#)
- [Software environment \(see page 9\)](#)
 - [Custom video driver and device tree \(see page 9\)](#)
 - [Serializer/De-serializer configuration \(see page 11\)](#)
 - [iRFE-Jetson \(see page 11\)](#)
 - [Strata \(see page 12\)](#)
- [Start-up \(see page 13\)](#)

1 Content of release package

This is the content of the Infineon-provided release package.

- **integration-carkit2C0-gmsl**
 - **scripts**
 - RSPS
 - Script to visualize raw data (with FFT calculation) recorded with v4l2-ctl (see below)
 - *capture_video0_4096x512.sh*
 - Script to capture the CSI-2 data stream on /dev/video0 with the format 4096x512 (see below)
 - tracing.sh
 - Enables and prints the kernel trace
 - **src**
 - jetson-config
 - Pinmux and GPIO DTSL files for flashing Nvidia Jetson
 - GMSL configuration driver
 - *demo_app*
 - iRFE driver with demo application
 - **strata-board-simulation**
 - Strata Jetson-side implementation
 - This is needed to interact with the system using Matlab/Python from an Ethernet-connected PC
 - **documentation**
 - The software user guide documentation

2 Features

CARKIT 2C0 software supports these use cases which are described further:

- Control and configuration interface to CARKIT2C0 via iRFE
- Raw data streaming over v4l2 (Video4Linux) framework
 - File (v4l2-utils)
 - Memory-mapped stream (v4l2 API)
- Control from Matlab/Python host PC

Both use cases are extensible by the user to support various measurement scenarios.

i The provided CARKIT 2C0 software acts as a starting point for development of a centralized Radar sensor. Please study carefully all provided examples and the accompanying documentation e.g. CTRX8191 User Manual.

2.1 Control and configuration interface

Using the Infineon-provided iRFE driver the user can configure and control both CTRX on the CARKIT 2C0, download RAM firmware and ramp designs.

The iRFE driver enables users to write their own custom C-applications for configuration and control of the CARKIT 2C0.

The iRFE provides access to all firmware functions of the CTRX. For further information on the configuration flow of CTRX and specific firmware functions please consult the CTRX8191 User Manual.

The iRFE uses the Linux *spidev* API to communicate over SPI with CTRX. The SPI traffic is tunneled over the GMSL2 link to the CARKIT 2C0.

This release includes example application to get you started with the iRFE under 'src\demo_app\boards\carkit_2c0\gmsl\example'

- This example shows the basic configuration flow of a cascaded CTRX system and will then enter a loop to trigger X amount of frames

The application can be compiled using the accompanying Makefile.

Please note: Application do not yet fully implement a productive Radar cycle. Infineon will provide an example in the next release.

2.2 Raw data streaming

Raw data streaming can be utilized over the Video4Linux (V4L2) API after the CARKIT 2C0 has been configured using custom iRFE code.

Infineon provides two examples to interact with the V4L2 API:

2.2.1 Using v4l2-ctl application

Using the *v4l2-ctl* application the user can stream data to a binary file.

The release includes an example script to start v4l2-ctl application and an example Python script to decode the binary data to a Matlab file.

To start v4l2-ctl in verbose mode:

```
v4l2-ctl -d /dev/video0 --set-fmt-video=width=4096,height=512,pixelformat=RG12 --set-ctrl  
bypass_mode=0 --stream-mmap --verbose  
  
v4l2-ctl -d /dev/video1 --set-fmt-video=width=4096,height=512,pixelformat=RG12 --set-ctrl  
bypass_mode=0 --stream-mmap --verbose
```

To start v4l2-ctl to stream to file:

```
v4l2-ctl -d /dev/video0 --set-fmt-video=width=4096,height=512,pixelformat=RG12 --set-ctrl  
bypass_mode=0 --stream-mmap --stream-to=ctrx0_raw.bin  
  
v4l2-ctl -d /dev/video1 --set-fmt-video=width=4096,height=512,pixelformat=RG12 --set-ctrl  
bypass_mode=0 --stream-mmap --stream-to=ctrx1_raw.bin
```

The resulting file will contain RAW data as outputted by Nvidia Video Input block.

To load the files use the Python scripts in '*scripts*' folder. These scripts demonstrate loading the data and doing some basic processing (FFT) on them.

2.2.2 Memory-mapped stream via a custom application

This release supports streaming data over a memory-mapped stream using the V4L2 API directly from a custom application.

The configuration of the streaming interface has to match the configured ramp design in terms of samples and ramps. Failure to do so will result in no data reception.

The provided device tree maps CTRX8191 Primary to */dev/video0* and CTRX8191 Secondary to */dev/video1*

As a starting point one can use the *V4LDataFrameCapture.cpp* class. In this class the following steps are shown

- Open and initialize */dev/videoX* device
- Configure device for Memory Mapped streaming of RAW data, using the supplied video format (Video Resolution = $4 \times \text{number_samples} \times \text{number_ramps}$)
- Get (*dequeue*) a frame from V4L2 buffer
- Remove 16-bit padding (*processRawData*) to mask ADC raw samples

The class will return a Frame in the following (native CTRX) format:

```
// The Frame is organized as follows:  
// | Entry1_Rx1, Entry1_Rx2, Entry1_Rx3, Entry1_Rx4, Entry2_Rx1, ..... , EntryN_Rx4 |  
// After retrieving one single receiver that leaves us with this layout for each of them:  
// Assuming nSamples = 4, nRamps = 2:  
// -----
```

```
// |           RAMP           |           RAMP           |  
// |SAMPLE SAMPLE SAMPLE SAMPLE|SAMPLE SAMPLE SAMPLE SAMPLE|
```

Additionally in the file CtrxTimeDataNode.cpp it is shown how to re-format the Frame into a common (Rx) (Ramps)(Samples) multi-dimensional array (*processFrameData*)

```
receivers[receiverIndex].ramps[ramp].samples[sample]
```

2.3 Control from Matlab host PC

Using the provided board simulator, Matlab can be used to control CARKIT 2C0 and receive data directly to Matlab over Ethernet via Strata.

Note that this use case is inherently limited by the Ethernet interface and thus will have limited performance for real-time data transfers. See chapter Strata.

3 Software and Hardware Pre-requisites

CARKIT 2C0 is intended to be used with

Nvidia Jetson AGX Orin development kit with release Jetson 35.4.1 (Jetpack 5.1.2)

<https://www.nvidia.com/en-us/autonomous-machines/embedded-systems/jetson-orin/>

<https://developer.nvidia.com/embedded/jetpack-sdk-512>

Note: Please execute following command to check current version of Jetson

```
cat /etc/nv_tegra_release
```

Should yield: # R35 (release), REVISION: 4.1, GCID: 33958178, BOARD: t186ref, EABI: aarch64, DATE: Tue Aug 1 19:57:35 UTC 2023

LI-AGO-MAX9296-ADPT V1.x deserializer board

Based on Analog® MAX9296A. Please get in contact with your local Infineon sales representative to obtain the board.

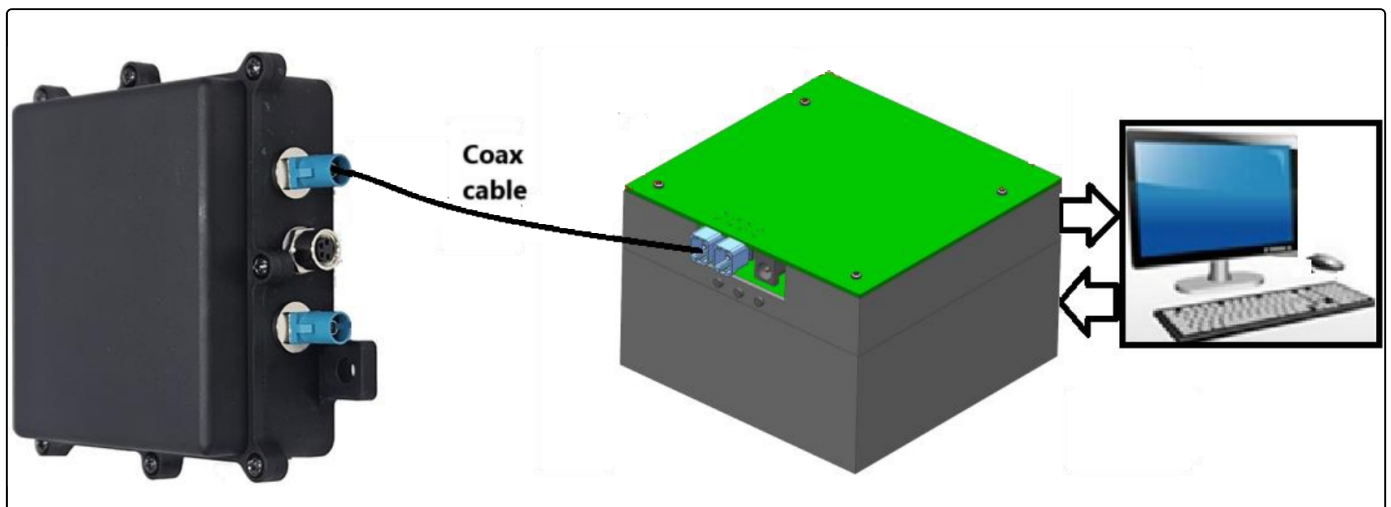
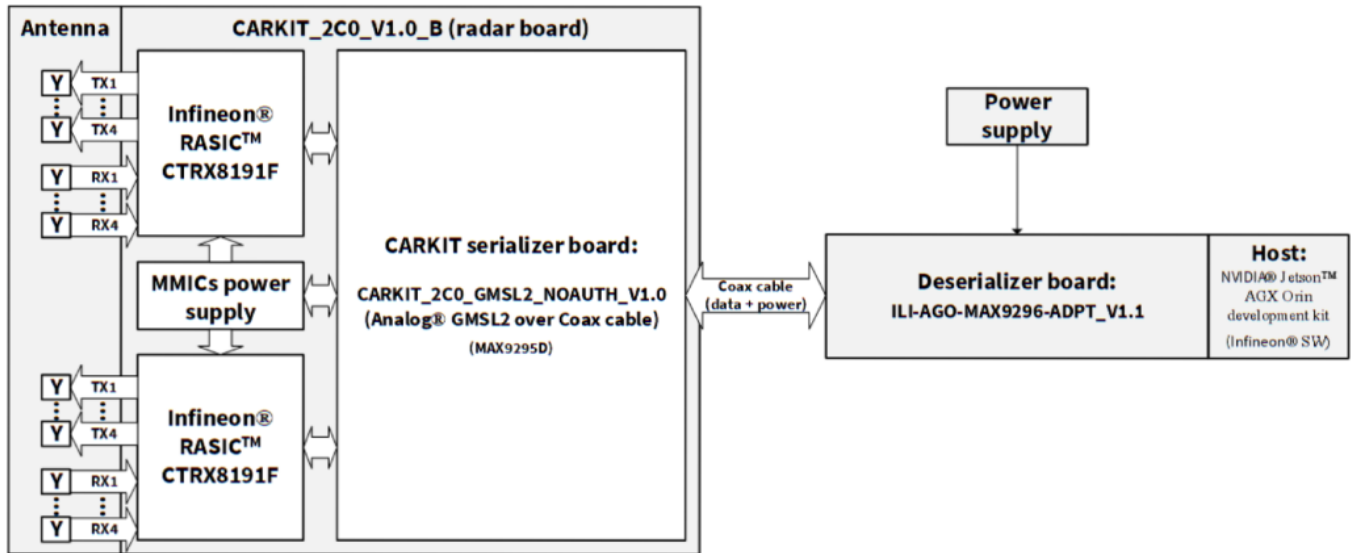
CARKIT 2C0 module

Including Coaxial cable

4 Hardware build-up

Below pictures gives an overview over the hardware architecture. Please refer to the Hardware User guide for detailed descriptions.

While the Deserializer board and CARKIT 2C0 sensor is supplied by Infineon together with cable and a case, Jetson Orin Development kit must be obtained through partners.



5 Software environment

Infineon provides the following software to operate CARKIT 2C0

- Custom video driver with custom device tree on Jetson Orin to receive CSI-2 data
- Configuration file to configure tunneling services over GMSL2
- iRFE-Jetson, a custom SPI and I2C driver to control CARKIT 2C0 from Jetson
- Strata, a MATLAB/Python interface to a host PC connected to Jetson through Ethernet

This chapter will provide an overview on how to do initial setup and operation of CARKIT 2C0

Important note: All software has been developed for and targets Jetpack 5.1.2. Please make sure you run Jetpack 5.1.2 on the Jetson before proceeding with setup and operation.

If not running Jetpack 5.1.2, please refer to Nvidia documentation (SDK Manager) on how to reflash the Jetson: <https://developer.nvidia.com/embedded/learn/get-started-jetson-agx-orin-devkit>

5.1 Custom video driver and device tree

The below steps need to be followed for the initial configuration of the jetson. To ease the process, Infineon can provide a Dockerfile that is setting up the build environment.

Prerequisites: WSL installation running Ubuntu-20.04

Infineon provides the pinmux file and GPIO configuration file in the packages which can be used to the bootloader as below:

```
# Copy pinmux file to bootloader directory
cp tegra234-mb1-bct-pinmux-p3701-0000-a04.dtsi $L4T/bootloader/t186ref/BCT/tegra234-mb1-
bct-pinmux-p3701-0000-a04.dtsi

# Copy GPIO configuration file
cp tegra234-mb1-bct-gpio-p3701-0000-a04.dtsi $L4T/bootloader/tegra234-mb1-bct-gpio-
p3701-0000-a04.dtsi
```

Infineon also provides a kernel patch in the package which can be applied as follows:

```
cd $L4T/source/public/
# Git commands are optional and just help to verify if patch was applied successfully
git init
git add .
git commit -m "Initial commit" # Optional

# Apply the patch
patch -p1 < /path/to/your/patch/carkit2C0-kernel.patch

# Verify patch was applied successfully
git status
```

To build and flash the kernel:

```
# Install missing packages
sudo apt-get install flex bison libssl-dev
```

```
# Build kernel with new configuration
cd $L4T/source/public/
./nvbuild.sh -o $PWD/kernel_out

# Copy the generated files
cp $L4T/source/public/kernel_out/arch/arm64/boot/dts/nvidia/tegra234-
p3701-0005-p3737-0000.dtb $L4T/kernel/dtb/
cp $L4T/source/public/kernel_out/arch/arm64/boot/Image $L4T/kernel/Image

# Flash the updated kernel and bootloader
cd $L4T/
sudo ./flash.sh jetson-agx-orin-devkit mmcblk0p1
```

The video driver is called radar.c and is then available as a built-in kernel module.

The current mode table is implemented and aligned with the device tree (Note: Width = 4*Number of Samples, Height=Number of Ramps)

Mode table:

```
static radar_reg *mode_table[] = {
    [RADAR_MODE_START_STREAM] = radar_start,
    [RADAR_MODE_STOP_STREAM] = radar_stop,
    [RADAR_MODE_1024x256_CROP_30FPS] = radar_1024x256_crop_30fps,
    [RADAR_MODE_2048x256_CROP_30FPS] = radar_2048x256_crop_30fps,
    [RADAR_MODE_4096x256_CROP_30FPS] = radar_4096x256_crop_30fps,
    [RADAR_MODE_8192x256_CROP_30FPS] = radar_8192x256_crop_30fps,
    [RADAR_MODE_16384x256_CROP_30FPS] = radar_16384x256_crop_30fps,
    [RADAR_MODE_1024x512_CROP_30FPS] = radar_1024x512_crop_30fps,
    [RADAR_MODE_2048x512_CROP_30FPS] = radar_2048x512_crop_30fps,
    [RADAR_MODE_4096x512_CROP_30FPS] = radar_4096x512_crop_30fps,
    [RADAR_MODE_8192x512_CROP_30FPS] = radar_8192x512_crop_30fps,
    [RADAR_MODE_16384x512_CROP_30FPS] = radar_16384x512_crop_30fps,
    [RADAR_MODE_1024x640_CROP_30FPS] = radar_1024x640_crop_30fps,
    [RADAR_MODE_2048x640_CROP_30FPS] = radar_2048x640_crop_30fps,
    [RADAR_MODE_4096x640_CROP_30FPS] = radar_4096x640_crop_30fps,
    [RADAR_MODE_8192x640_CROP_30FPS] = radar_8192x640_crop_30fps,
    [RADAR_MODE_16384x640_CROP_30FPS] = radar_16384x640_crop_30fps,
    [RADAR_MODE_1024x1024_CROP_30FPS] = radar_1024x1024_crop_30fps,
    [RADAR_MODE_2048x1024_CROP_30FPS] = radar_2048x1024_crop_30fps,
    [RADAR_MODE_4096x1024_CROP_30FPS] = radar_4096x1024_crop_30fps,
    [RADAR_MODE_8192x1024_CROP_30FPS] = radar_8192x1024_crop_30fps,
    [RADAR_MODE_16384x1024_CROP_30FPS] = radar_16384x1024_crop_30fps,
};
```

For the libgpod, please follow the steps below:

```
# Update package repository
sudo apt update

# Install prerequisites
sudo apt install git build-essential autoconf automake autoconf-archive libtool pkg-config

# Clone libgpod sources at correct branch
git clone --branch v1.6.x https://git.kernel.org/pub/scm/libs/libgpod/libgpod.git
```

```
cd libgpod

# Configure installation
./autogen.sh --enable-tools=yes

# Build
make -j$(nproc)

# Install
sudo make install
```

For v4l-utils:

```
sudo apt install libi2c-dev v4l-utils
```

5.2 Serializer/De-serializer configuration

The driver for the serializer/deserializer is provided under max929x.c file in the package. It is available as a built-in kernel module.

5.3 iRFE-Jetson

The iRFE-Jetson is a comprehensive software integration package for the CARKIT 2C0 radar system, connected to the Nvidia Jetson platform via GMSL2 (SerDes) technology, as shown in the [Hardware build-up](#)¹ section. The iRFE-Jetson provides control and interface capabilities for both CTRX8191 radar chips on the CARKIT 2C0 module, enabling users to write custom C applications with the necessary configuration and control of the CARKIT 2C0 module.

The iRFE-Jetson includes:

- iRFE-driver (core component)
 - Access to firmware commands
 - Convenience functions
- Platform Abstraction Layer for the Nvidia Jetson
 - Communication interfaces
 - SPI, I2C, GPIO
 - Timing services
 - Get current time, delay, and timeout
- Hardware-specific implementations for the CARKIT 2C0
 - GPIO pin definitions, SPI configuration, device tree integration

The C-code example application is located in the 'src/demo_app\boards\carkit_2c0\gmsl\example' directory. The application shows the complete initialization, configuration, and operation of the CARKIT 2C0 module.

It includes:

- Complete system initialization (from power-on to operational state)
- Coordinated radar operations (with synchronized measurement cycles)
- Power management (with low-power modes between measurements)
- PMIC configuration (for proper voltage supply management)
- TX calibration procedures (for optimal radar performance)

1. <https://confluencewikiprod.intra.infineon.com/spaces/ASRSSET/pages/2360607410/CARKIT+2C0+User+Guide#CARKIT2C0UserGuide-Hardwarebuild-up>

To build and execute the example, execute the following steps:

```
# Enter the example directory
cd <root_dir>/src/demo_app/boards/carkit_2c0/gmsl/example

# Build using make
make

# Execute the example
./bin/carkit2c0_gmsl_example
```

The user can use the existing example application as a template to develop a custom application for a specific use case.

5.4 Strata

To control the Jetson from Matlab, at least Strata 2.9.x is needed. For details of the Strata interface, please refer to the SW user guide provided with the Strata package under <StrataBasePath\documentation>.

In a typical scenario, Jetson should be configured for IP address 169.254.1.101 and the Matlab host PC for IP address 169.254.1.1.

The Jetson side of the Matlab interface is provided as a Docker container. The Docker container must be build and can then be launched.

1. Once the initialization (see *Start System*) is successful, start the board simulator. Board simulator can be launched using the following steps:
 - I. Navigate to the *strata-board-simulation* folder in the Jetson
 - II. Execute: *docker load -i strata-board-simulator-carkit-2c0.tar.gz*
 - III. Execute: *docker run -i --rm -p 55055:55055 -p 55056:55056 --privileged board-simulator-carkit-2c0*
2. On the host pc:
 - I. When connected to the Jetson via SSH, ensure tunneling of local ports 55055 & 55056 via SSH to the Jetson.

Strata on Host side comes with an example CARKIT2C0_example_jetson.m (<strataBasePath\matlab\examples>) that can directly be used with the board simulator.

6 Start-up

Hardware

Before powering the Jetson make sure the Deserializer board is powered. The Deserializer board will provide power to the CARKIT 2C0 module over the connected coax cable (Power over GMSL).

Post power up check if the CARKIT 2C0 is detectable over i2c via the i2c detect commands:

```
i2cdetect -r -y 2
```

For CARKIT 2C0 serializer, the i2c device address is 0x40.

Software

After first setup and flash of the kernel module of the jetson no further steps are required to start post power up, all the relevant modules are available as built-in kernel modules currently.

Trademarks

All referenced product or service names and trademarks are the property of their respective owners.

Edition yyyy-mm-dd

Published by

Infineon Technologies AG

81726 Munich, Germany

© 2023 Infineon Technologies AG.

All Rights Reserved.

Do you have a question about this document?

Email: erratum@infineon.com

Document reference

AppNote Number

IMPORTANT NOTICE

The information contained in this application note is given as a hint for the implementation of the product only and shall in no event be regarded as a description or warranty of a certain functionality, condition or quality of the product. Before implementation of the product, the recipient of this application note must verify any function and other technical information given herein in the real application. Infineon Technologies hereby disclaims any and all warranties and liabilities of any kind (including without limitation warranties of non-infringement of intellectual property rights of any third party) with respect to any and all information given in this application note.

The data contained in this document is exclusively intended for technically trained staff. It is the responsibility of customer's technical departments to evaluate the suitability of the product for the intended application and the completeness of the product information given in this document with respect to such application.

For further information on the product, technology, delivery terms and conditions and prices please contact your nearest Infineon Technologies office (www.infineon.com).

WARNINGS

Due to technical requirements products may contain dangerous substances. For information on the types in question please contact your nearest Infineon Technologies office.

Except as otherwise explicitly approved by Infineon Technologies in a written document signed by authorized representatives of Infineon Technologies, Infineon Technologies' products may not be used in any applications where a failure of the product or any consequences of the use thereof can reasonably be expected to result in personal injury.